

Outlines

- Introduction to Constructor
- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Destructor
- Constructor VS Destructor

Constructor

Definition: A constructor is a special member function that is automatically called when an object is created.

Typically used to initialize member variables of the class to appropriate default or user-provided values, or to do any setup steps necessary for the class to be used (e.g. open a file or database).

Unlike normal member functions, constructors have specific rules for how they must be named.

- Constructors must have the same name as the class (with the same capitalization)
- Constructors have no return type (not even void)

Key Points/Features:

- Constructor has same name as the class itself.
- Constructors don't have return type, not even '**void**'.
- A constructor is automatically called when an object is created.
- It is usually placed in public section of the class.
- Constructor can be overloaded.
- If we do not specify a constructor, C++ compiler generates a default constructor for object (expects no parameters and has an empty body).

Syntax

```
class ClassName {  
public:  
    ClassName(); // Constructor declaration  
};
```

```
ClassName::ClassName() {  
    // Constructor definition  
}
```

Example

```
#include <iostream>
using namespace std;
class MyClass { // The class
public: // Access specifier
MyClass() { // Constructor
cout << "Hello World!";
}
};
int main() {
MyClass myObj; // Create an object of
MyClass (this will call the constructor)
return 0;
}
```

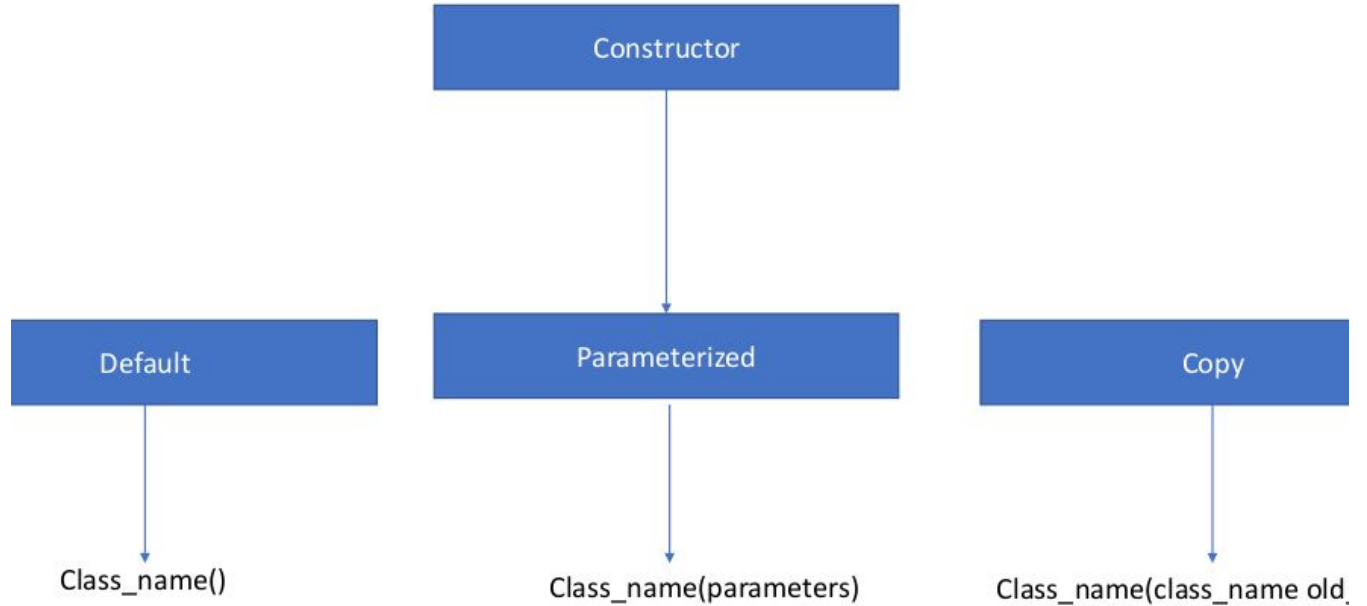
```
#include <iostream>
using namespace std;
class MyClass { // The class
public: // Access specifier
MyClass(); // Constructor declaration
};

MyClass:: MyClass () {
    cout << "Hello World!";
}

int main() {
MyClass myObj; // Create an object of
MyClass (this will call the constructor)
return 0;
}
```

Types of Constructors

- Default
- Parameterized
- Copy



Default Constructor

- Default constructor is the constructor which doesn't take any argument (or has parameters that all have default values)

```
#include <iostream>
using namespace std;
class Number
{
public:
    int a, b;
    // Default Constructor
    Number() {
        a = 10;
        b = 20;
    }
};

int main()
{
    // Default constructor called automatically
    // when the object is created
    Number n;
    cout << "a: " << n.a << "b: " << n.b;
    return 0;
}
```

Default Parameters in Constructor

Usage:

- Default parameters in constructors allow for the creation of objects with flexible initialization.
- They simplify object creation by providing a default value if no argument is provided explicitly.

Benefits:

- Avoids the need for multiple constructors with different parameter lists for common initialization scenarios.
- Provides a convenient way to specify default values for object initialization.

```
#include <iostream>
using namespace std;
class Number {
private:
    int num;

public:
    // Constructor with default
    argument
    Number(int n = 0) {
        num = n;
    }

    void display() {
        cout << "Number: " << num <<
        endl;
    }
};
```

```
int main() {
    Number n1; // Default constructor
    called with default argument
    n1.display(); // Output: Number: 0

    Number n2(5); // Custom value
    n2.display(); // Output: Number: 5

    Number n3(10); // Custom value
    n3.display(); // Output: Number: 10

    return 0;
}
```

Implicit Constructor

If a class has no constructors, C++ will automatically generate a public default constructor that allows to create an object without the arguments.

This is sometimes called an implicit constructor (or implicitly generated constructor).

If your class has any other constructors, the implicitly generated constructor will not be provided.

```
#include <iostream>
using namespace std;

class MyClass {
private:
    int value;

public:
    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    MyClass obj;
    obj.display(); // Will compile successfully
    return 0;
}
```

Parameterized Constructor

- The constructors that can take arguments are called parameterized constructors.
- To create a parameterized constructor, simply add parameters to it the way you would to any other function.
- When you define the constructor's body, use the parameters to initialize the object.
- Constructor function can be defined as inline function.

Uses of Parameterized constructor:

- It is used to initialize the various data elements of different objects with different values when they are created.
- It is used to overload constructors.

Syntax

```
class ClassName {  
    public:  
        ClassName (Type arg1, Type arg2); // Parameterized constructor  
        declaration  
};  
  
ClassName::ClassName (Type arg1, Type arg2) {  
    // Constructor definition  
}
```

```
#include <iostream>
using namespace std;

class Number {
public:
    int a, b;

    // Parameterized Constructor
    Number(int x, int y) {
        a = x;
        b = y;
    }

    void display() {
        cout << "a: " << a << " b: " << b <<
endl;
    }
};
```

```
int main() {
    // Parameterized constructor
    called when the object is
    created with arguments
    Number n(30, 40);
    cout << "Using parameterized
constructor:" << endl;

    n.display();

    return 0;
}
```

Default & Parameterized Example

```
#include <iostream>
using namespace std;
class Number {
public:
    int a, b;
    // Default Constructor
    Number() {
        a = 10;
        b = 20;
    }
    // Parameterized Constructor
    Number(int x, int y) {
        a = x;
        b = y;
    }
    void display() {
        cout << "a: " << a << " b: " << b <<
endl;
    }
};
```

```
int main() {
    // Default constructor called
    automatically when the object is created
    Number n1;
    cout << "Using default constructor:" <<
endl;
    n1.display();

    // Parameterized constructor called
    when the object is created with arguments
    Number n2(30, 40);
    cout << "Using parameterized
constructor:" << endl;
    n2.display();

    return 0;
}
```

Copy Constructor

Definition: A constructor that initializes an object using another object of the same class.

Purpose: Create a copy of an existing object.

A copy constructor is a special type of constructor used to create a new object as a copy of an existing object.

A reference variable is used as an argument to the copy constructor.

Key Point: We cannot pass the argument by value to a copy constructor.

Reason:

- Passing by value creates a copy, invoking the copy constructor repeatedly.
- This leads to an infinite loop and eventually runs out of memory.

Solution: Pass the argument by reference to avoid creating copies.

```

#include <iostream>
using namespace std;

class Number {
public:
    int a, b;
    // Default Constructor
    Number() {
        a = 10;
        b = 20;
    }

    // Copy Constructor
    Number(Number &obj) {
        a = obj.a;
        b = obj.b;
    }

    void display() {
        cout << "a: " << a << " b: " << b <<
endl;
    }
};

```

```

int main() {
    // Default constructor called automatically
when the object is created

    Number n1;
    cout << "Using default constructor:" <<
endl;
    n1.display();

    // Copy constructor called when a copy of
the object is created

    Number n2(n1);
    cout << "Using copy constructor:" << endl;
    n2.display();

    Number n3 = n1; // Alternate calling process
    cout << "Using copy constructor:" << endl;
    n3.display();

    return 0;
}

```

Summary

Default Constructor: No parameters, initializes members with default values.

Parameterized Constructor: Takes parameters to initialize members with specific values.

Copy Constructor: Takes a reference to another object of the same class, used to create a copy of an existing object.

Overloading Constructor

When a class contains more than one constructor that is called overloading constructor.

Multiple constructors can be declared in the same class.

Like functions, constructors can also be overloaded.

```

#include <iostream>
using namespace std;
class Number {
public:
    int a, b;
    // Default Constructor
    Number() {
        a = 10;
        b = 20;
    }
    // Parameterized Constructor
    Number(int x, int y) {
        a = x;
        b = y;
    }
    // Copy Constructor
    Number(const Number &obj) {
        a = obj.a;
        b = obj.b;
    }
    void display() const {
        cout << "a: " << a << " b: " << b << endl;
    }
};

```

```

int main() {
    // Using Default Constructor
    Number n1;
    cout << "Using default constructor:" << endl;
    n1.display();

    // Using Parameterized Constructor
    Number n2(30, 40);
    cout << "Using parameterized constructor:" <<
endl;
    n2.display();

    // Using Copy Constructor
    Number n3(n2);
    cout << "Using copy constructor:" << endl;
    n3.display();

    return 0;
}

```

Q. Constructor in a Class: Mandatory and Multiple Constructors

Q. Is it mandatory to have a constructor in a class? can you have more than one constructor in a class, explain it with example? (Jan, 2023)

It's not mandatory to have a constructor in a class. If you don't define any constructor, the compiler provides a default constructor.

You can have more than one constructor in a class, which is known as constructor overloading. Each constructor can have a different set of parameters. For instance, default constructor, parameterized constructor, and copy constructor have different set of parameters.

Destructor

Definition and Purpose:

- A special member function executed when an object is destroyed.
- Used to perform cleanup tasks, such as releasing memory or other resources allocated during the object's lifetime.

Syntax:

- Named with a tilde (~) followed by the class name.
- Example: **~ClassName()**.

```
class MyClass {  
public:  
    ~MyClass() {  
        // Destructor code here  
    }  
};
```

Destructor contd.

Characteristics:

- **No Arguments:** Destructors cannot take parameters.
- **No Return Type:** Destructors do not return any value, not even `void`.
- **Cannot Be Overloaded:** Each class can have only one destructor.
- **Special Member Function:** Like constructors, destructors are special member functions of a class.

Automatic Invocation:

- Called automatically when an object goes out of scope or is explicitly deleted using the `delete` keyword.

Order of Destruction:

- Destructors are called in the reverse order of construction.

Memory Management:

- Essential for classes that allocate resources such as dynamic memory, file handles, or database connections.
- Ensures resources are properly released when the object is destroyed.

Explicit Call:

- Generally, destructors should not be called explicitly.
- The destructor can be called explicitly, but this is not recommended as it can lead to undefined behavior.

```
MyClass m(5);  
m.~MyClass(); // Explicit call (not recommended)
```

```
#include <iostream>
using namespace std;
class Number {
public:
    int a, b;
    // Constructor
    Number(int x, int y) {
        a = x;
        b = y;
        cout << "Constructor called, values initialized
to " << a << " and " << b << endl;
    }
    // Destructor
    ~Number() {
        cout << "Destructor called, Number object with
values " << a << " and " << b << " is being
destroyed" << endl;
    }

    void display(){
        cout << "a: " << a << " b: " << b << endl;
    }
};
```

```
int main() {
    Number n(5, 10); // Constructor called
    n.display();

    // Destructor will be called
    automatically when 'n' goes out of scope
    return 0;
}
```

Try it!!!

Q. Write a program to show that the execution of destructor is in reverse order than that of execution of constructor. Also display the corresponding object id (Dec, 2018).

Features of Destructor and Comparison with Constructor (Jan, 2018)

Features of Destructor:

1. **Releases resources:** Deallocates memory, closes files, or releases other resources acquired during object's lifetime.
2. **Called when object goes out of scope:** Automatically invoked when an object is no longer in use.
3. **No return type or arguments:** Like constructors, destructors don't have a return type and cannot take arguments.
4. **Clean-up tasks:** Performs cleanup operations, ensuring proper resource management.

Difference from Constructor

- Constructors are called when objects are created, while destructors are called when objects are destroyed.
- Constructors initialize object state, while destructors perform cleanup tasks.
- Constructors can be overloaded (i.e. multiple constructors are possible), but only one destructor can exist in a class.



Any Queries?